Association Rule Mining—FP Growth

BY

DR. ANUPAM GHOSH

DATE: 15TH APRIL, 2026

Introduction

- At each step, candidate sets have to be built.
- To build the candidate sets, the algorithm has to repeatedly scan the database.

- These two properties inevitably make the algorithm slower. To overcome these redundant steps, a new association-rule mining algorithm was developed named Frequent Pattern Growth Algorithm.
- It overcomes the disadvantages of the Apriori algorithm by storing all the transactions in a Tree Data Structure.

Transaction ID	Items
T1	{E,K,M,N,O,Y}
T2	{D,E,K,N,O,Y}
T3	{A,E,K,M}
T4	{C,K,M,U,Y}
T5	{C,E,I,K,O,O}

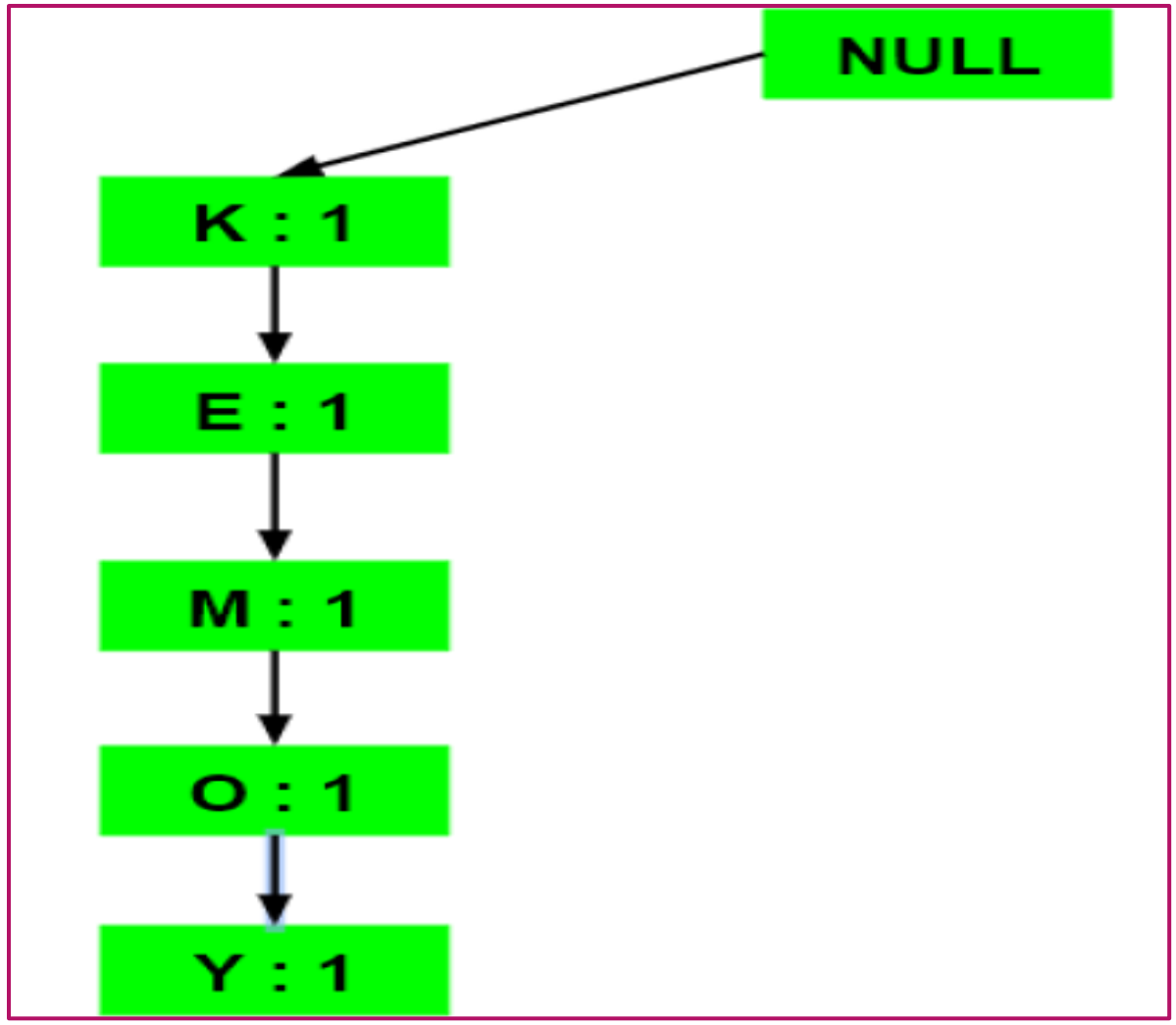
Let the minimum support be 3. A **Frequent Pattern set** is built which will contain all the elements whose frequency is greater than or equal to the minimum support. These elements are stored in descending order of their respective frequencies. After insertion of the relevant items, the set L looks like this:-

L = {K : 5, E : 4, M : 3, O : 3, Y : 3}

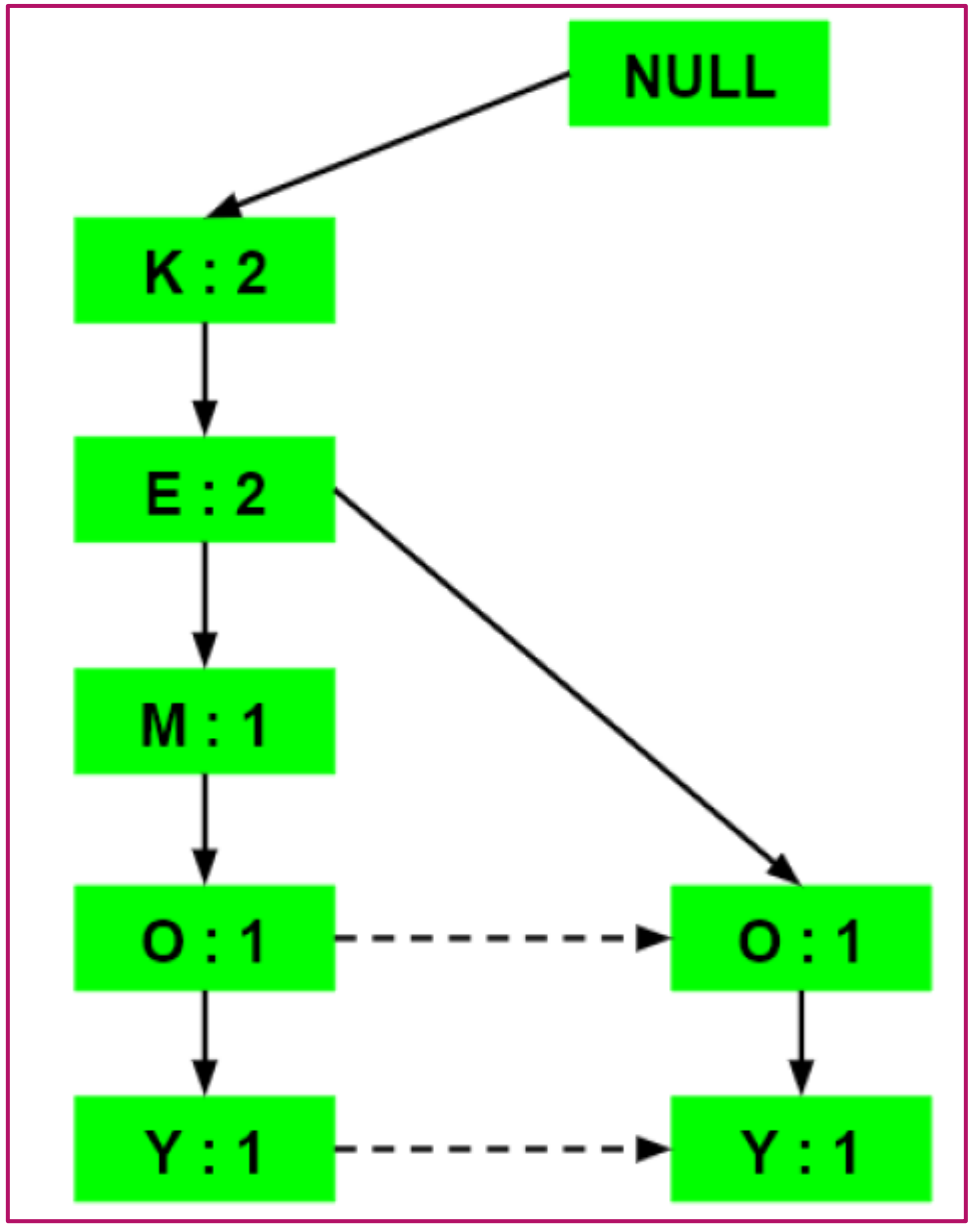
Item	Frequency
A	1
C	2
D	1
E	4
I	1
K	5
M	3
N	2
O	3
U	1
Y	3

Transaction ID	Items	Ordered-Item Set
T1	{E,K,M,N,O,Y}	{K,E,M,O,Y}
T2	{D,E,K,N,O,Y}	{K,E,O,Y}
T3	{A,E,K,M}	{K,E,M}
T4	{C,K,M,U,Y}	{K,M,Y}
T5	{C,E,I,K,O,O}	{K,E,O}

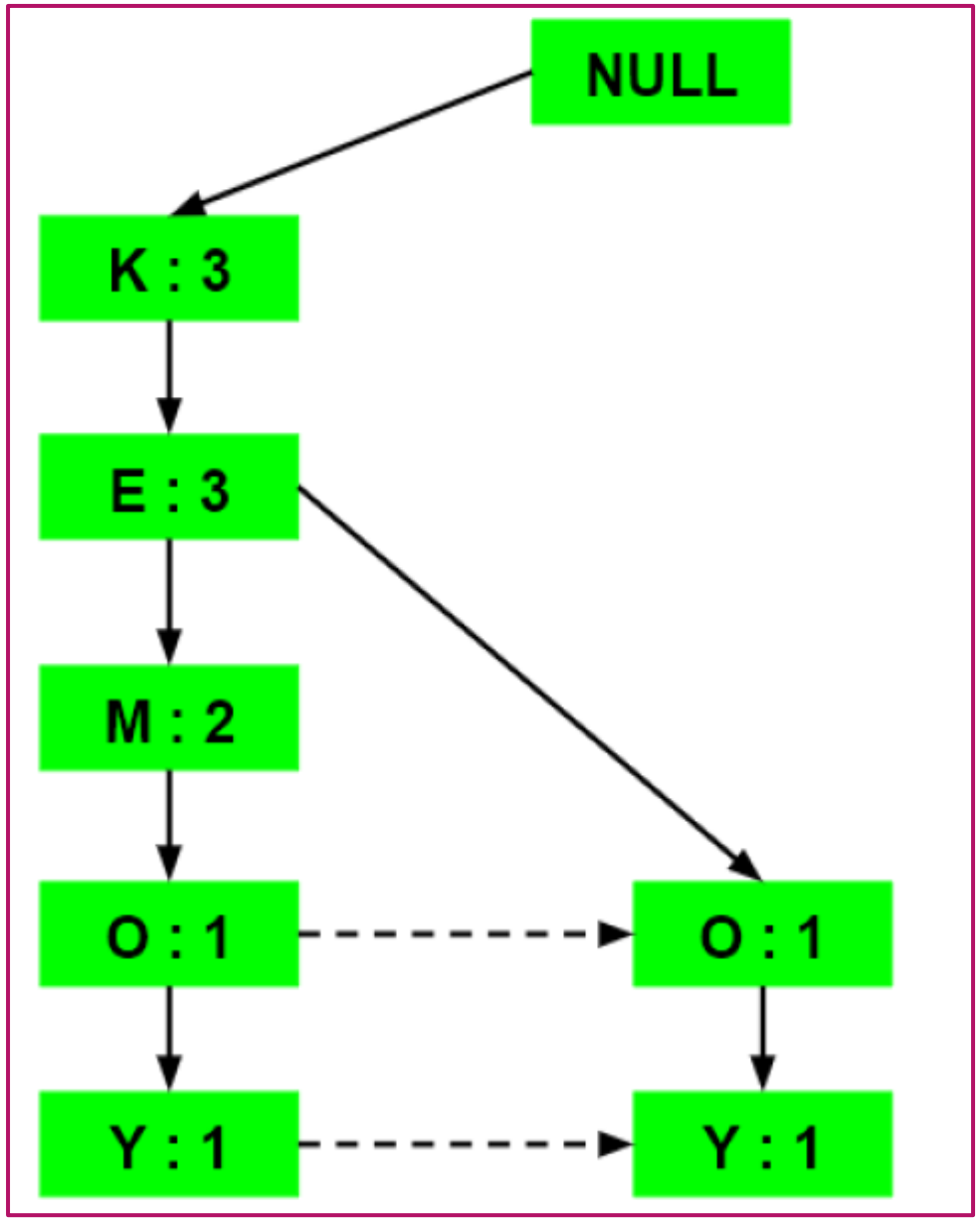
a) **Inserting the set {K, E, M, O, Y}:**



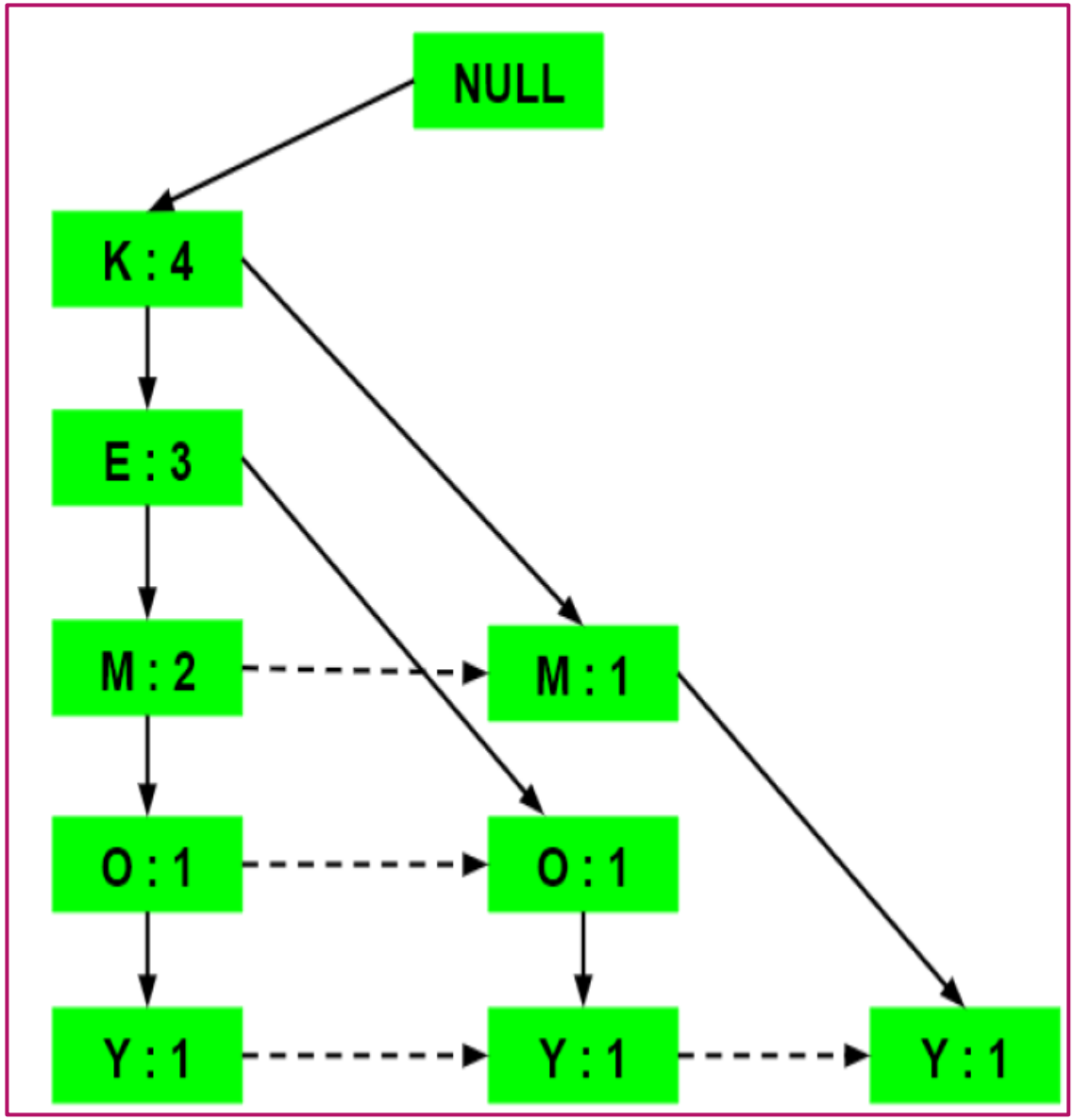
Inserting the set {K, E, O, Y}



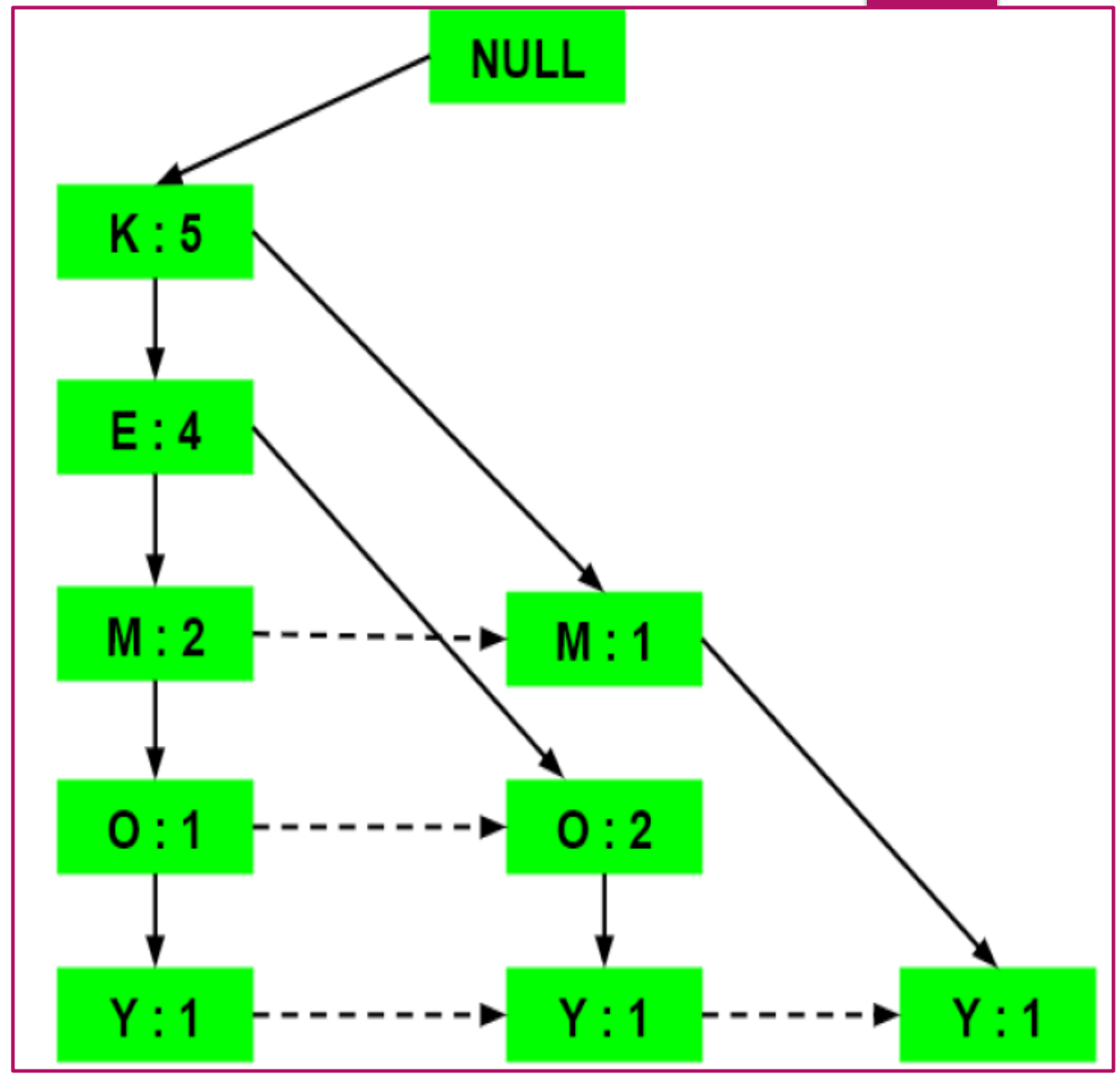
Inserting the set {K, E, M}



Inserting the set {K, M, Y}



Inserting the set {K, E, O}



Items	Conditional Pattern Base
Y	{{K,E,M,O : 1}, {K,E,O : 1}, {K,M : 1}}
O	{{K,E,M : 1}, {K,E : 2}}
M	{{K,E : 2}, {K : 1}}
E	{K : 4}
K	

Items	Conditional Pattern Base	Conditional Frequent Pattern Tree
Y	{{K,E,M,O : 1}, {K,E,O : 1}, {K,M : 1}}	{K : 3}
O	{{K,E,M : 1}, {K,E : 2}}	{K,E : 3}
M	{{K,E : 2}, {K : 1}}	{K : 3}
E	{K : 4}	{K : 4}
K		

Items	Frequent Pattern Generated
Y	{<K,Y : 3>}
O	{<K,O : 3>, <E,O : 3>, <E,K,O : 3>}
M	{<K,M : 3>}
E	{<E,K : 3>}
K	

- For each row, two types of association rules can be inferred for example for the 1rst row which contains the element, the rules K -> Y and Y -> K can be inferred.
- To determine the valid rule, the confidence of both the rules is calculated and the one with confidence greater than or equal to the minimum confidence value is retained.
- S; A; A->(S-A)
- <K,Y>--- {k},{y}

K->Y; Y->K
<K, O>; <E,O>;
<E,K,O>--- {E},{K},{O}, {E,K},{E,O},{K,O}, {E,K,O}

{E}->{K,O}; {K}->{E,O}; {O}->{E,K}; {O,K}->{E}
{E,K}->{O}; {E,O}->{K};

<K,M>; <E,K>

Items	Frequent Pattern Generated
Y	{<K,Y : 3>}
O	{<K,O : 3>, <E,O : 3>, <E,K,O : 3>}
M	{<K,M : 3>}
E	{<E,K : 3>}
K	

- <k,y> ; non empty subsets: {k}, {y}, {k,y}; the rules: {k}->{y}; {y}->{k}
- Theory: if l is a frequent pattern and s is the non-empty subset; then rule will be constructed using following formula
- s -> (l-s)
- <k,O> .... {K}->{O}; {O}->{k}
- <E,O>.... {E} -> {O}; {O}->{E}
- <E,K,O>.....non-empty sub sets: {E}, {k}, {O}, {E,k}, {E,O}, {K,O}, {E,K,O}

Rules: {E}-> {K,O}; {K}->{E,O}; {O}->{E,K}; {E,k}->{O}; {E,O}->{K}; {K,O}->{E}; {E,K,O}->NULL

- Confidence of rule (X->Y) = Support count (XUY) / Support count (X)

Algorithm: **FP_growth.** Mine frequent itemsets using an FP-tree by pattern fragment growth.

Input:

- D, a transaction database;
- *min_sup*, the minimum support count threshold.

Output: The complete set of frequent patterns.

Method:

1. The FP-tree is constructed in the following steps:

   (a) Scan the transaction database D once. Collect F, the set of frequent items, and their support counts. Sort F in support count descending order as L, the *list* of frequent items.

   (b) Create the root of an FP-tree, and label it as "null." For each transaction *Trans* in D do the following.
   Select and sort the frequent items in *Trans* according to the order of L. Let the sorted frequent item list in *Trans* be $[p|P]$, where p is the first element and P is the remaining list. Call insert_tree($[p|P]$, T), which is performed as follows. If T has a child N such that $N.item\text{-}name = p.item\text{-}name$, then increment N's count by 1; else create a new node N, and let its count be 1, its parent link be linked to T, and its node-link to the nodes with the same *item-name* via the node-link structure. If P is nonempty, call insert_tree(P, N) recursively.

2. The FP-tree is mined by calling FP_growth(*FP_tree*, *null*), which is implemented as follows.

```
procedure FP_growth(Tree, α)
(1)   if Tree contains a single path P then
(2)       for each combination (denoted as β) of the nodes in the path P
(3)           generate pattern β ∪ α with support_count = minimum support count of nodes in β;
(4)   else for each a_i in the header of Tree {
(5)       generate pattern β = a_i ∪ α with support_count = a_i.support_count;
(6)       construct β's conditional pattern base and then β's conditional FP_tree Tree_β;
(7)       if Tree_β ≠ ∅ then
(8)           call FP_growth(Tree_β, β); }
```

It will be Continued….